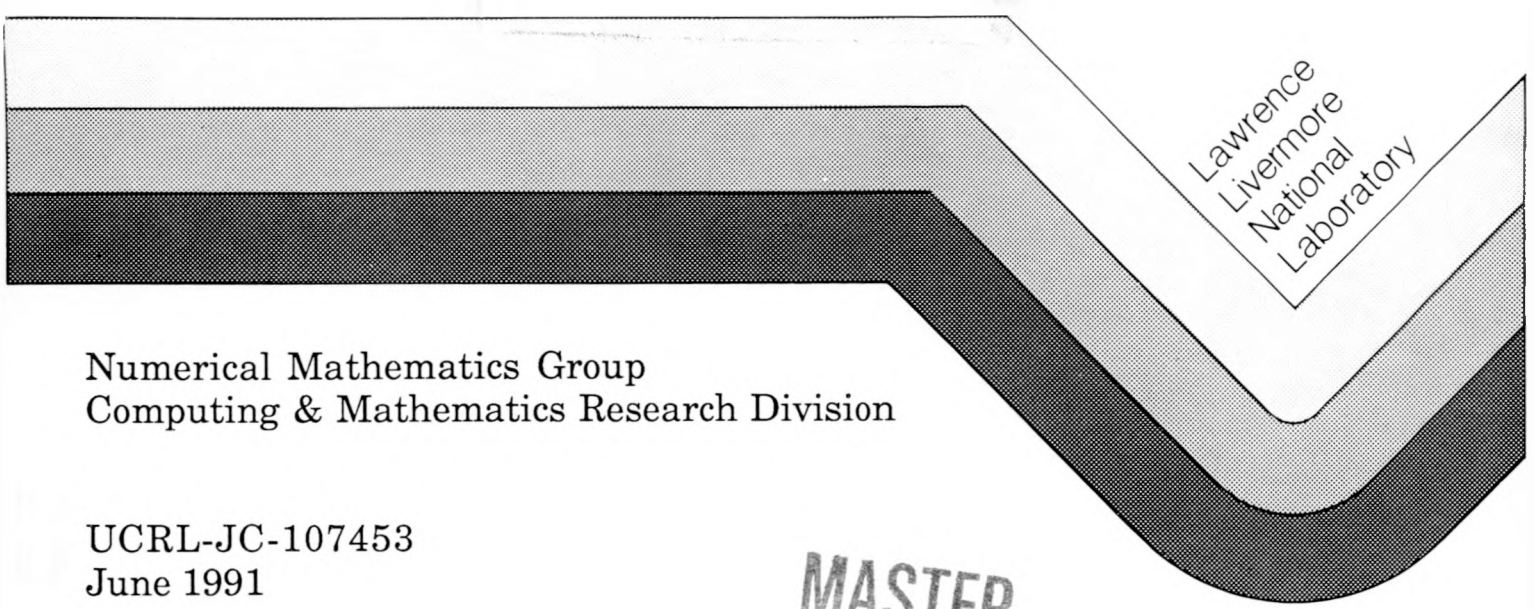
The Parallel BFGS Method for Unconstrained Minimization

Charles H. Still
Lawrence Livermore National Laboratory



Numerical Mathematics Group
Computing & Mathematics Research Division

UCRL-JC-107453
June 1991

PREPRINT

This is a preprint of a paper submitted to Conference: The Sixth Distributed Memory Computing Conference; Portland, OR; April 28 - May 1, 1991. Since changes may be made before publication, this preprint is made available with the understanding that it will not be cited or reproduced without the permission of the author.

UCRL-JC--107453

DE91 015784

The Parallel BFGS Method for Unconstrained Minimization*

C. H. Still
Numerical Mathematics Group
Lawrence Livermore National Laboratory
Livermore, CA 94550

Abstract

In this paper, we present a parallel formulation of the BFGS method for unconstrained minimization obtained by decomposing the iteration and update equations with an orthonormal basis. To aid in evaluating the performance of this method, numerical results are presented from experiments run on an nCUBE/1 hypercube and a Symult S2010. Finally, we present the theoretical performance analysis of the algorithm on these machines solving the test problems to substantiate the numerical testing.

1 Introduction

Consider the twice continuously differentiable function $f : \Omega \subset \mathrm{R}^n \rightarrow \mathrm{R}$. We wish to solve the problem

$$\min_{x \in \Omega} f(x),$$

assuming that there is some $x_* \in \Omega$ such that $f(x_*) \leq f(x)$ for all $x \in \Omega$. It is easy to demonstrate functions f which require a great deal of time and resources to evaluate, even when the problem dimension n is not very large. Even worse are problems where the function evaluation involves the solution of a system of differential equations, or a large scale simulation. Of

*This work was supported in part by the Applied Mathematical Sciences subprogram of the Office of Energy Research, U.S. Department of Energy, by Lawrence Livermore National Laboratory under contract W-7405-ENG-48.

the types of parallel algorithms which can be used efficiently to solve these larger problems, Byrd, Schnabel, and Shultz have studied secant methods which try to do parallel function evaluations by predicting possible next iterates [1]. Another approach is to decompose the iterations into their coordinate components.

Broyden's inverse Method has a parallel analog that can be implemented efficiently on a distributed memory machine which is based on the decomposition of an iteration into its coordinate components [7]. Unfortunately, the inverse Broyden method has several disadvantages involving the types of problems that it can solve [2]. Thus, we now wish to develop a parallel method with more applicability. The most widely used method for unconstrained minimization is the Positive Definite Secant Method, or BFGS method after its discoverers, Broyden, Fletcher, Goldfarb, and Shanno. We will show that the inverse BFGS method has an efficient parallel version.

The inverse secant method has the following form:

$$x_+ = x_c - H_c g_c \tag{1}$$
$$H_+ = H_c - uv^T. \tag{2}$$

Now we introduce the notation that $s = x_+ - x_c$ and $y = g_+ - g_c$; this is standard with the exception that the subscripts have been dropped (s_c has become s and y_c has become y). The notation is unambiguous since we will never need s_+ or y_+. Using the above notation, we can write the BFGS update for the inverse secant method in the following way:

$$H_+ = H_c + \frac{(s - H_c y)s^T + s(s - H_c y)^T}{y^T s} - \frac{< s - H_c y, y > ss^T}{(y^T s)^2}. \tag{3}$$

If we choose an orthonormal basis $\{b_i\}$ for the space $\mathbf{R}^n$, we can rewrite (1) as

$$\xi_i^+ = \xi_i^c - \sum_{j=1}^{n} \gamma_j^c b_i^T h_j^c,$$

where $H_c = [h_1^c, h_2^c, \ldots, h_n^c]$ is listed by its columns. This formula can be rewritten, in turn, as

$$\xi_i^+ = \xi_i^c - \sum_{j=1}^{n} \gamma_j^c \beta_{ij}^c, \tag{4}$$

where we define $\beta_{ij}^c = b_i^T h_j^c$. We now seek a componentwise form for (3), i.e. an update strategy for β_{ij}^c suitable for use on a distributed memory machine.

First, we write (3) in its columnar form

$$\begin{aligned} h_j^+ &= h_j^c + \sigma_j \frac{(s - H_c y)}{y^T s} + [s - H_c y]_j \frac{s}{y^T s} \\ &\quad -\sigma_j \frac{(s - H_c y)^T y}{(y^T s)^2}, \end{aligned} \tag{5}$$

where $[s - H_c y]_j$ denotes the j^{th} entry in the vector $s - H_c y$. Note, by expanding $s - H_c y$ in the basis $\{b_i\}$, we have

$$\begin{aligned} [s - H_c y]_j &= \left[\sum_{k=1}^n \sigma_k b_k - \sum_{k=1}^n \eta_k h_k^c\right]_j \\ &= \left[\sum_{k=1}^n (\xi_k^+ - \xi_k^c) b_k - (\gamma_k^+ - \gamma_k^c) h_k^c\right]_j \\ &= b_j^T \left(\sum_{k=1}^n (\xi_k^+ - \xi_k^c) b_k - (\gamma_k^+ - \gamma_k^c) h_k^c\right) \\ &= \xi_j^+ - \xi_j^c - \sum_{k=1}^n (\gamma_k^+ - \gamma_k^c) b_j^T h_k^c \\ &= \left(\xi_j^c - \sum_{k=1}^n \gamma_k^c \beta_{jk}^c\right) - \xi_j^c \\ &\quad - \sum_{k=1}^n (\gamma_k^+ - \gamma_k^c) \beta_{jk}^c \\ &= -\sum_{k=1}^n \gamma_k^+ \beta_{jk}^c. \end{aligned} \tag{6}$$

Furthermore, by a similar computation, we have that

$$b_i^T (s - H_c y) = -\sum_{k=1}^n \gamma_k^+ \beta_{ik}^c, \tag{7}$$

and

$$\begin{aligned} (s - H_c y)^T y &= y^T (s - H_c Y) \\ &= \sum_{i=1}^n \eta_i b_i^T (s - H_c y) \end{aligned} \tag{8}$$

$$= -\sum_{i=1}^{n}\sum_{k=1}^{n}\eta_i\gamma_k^+\beta_{ik}^c$$
$$= -\sum_{i=1}^{n}\sum_{k=1}^{n}(\gamma_i^+ - \gamma_i^c)\gamma_k^+\beta_{ik}^c. \quad (9)$$

Premultiplying (5) by b_i^T and substituting (6) and (7), we obtain the desired update for β_{ij}^+:

$$\beta_{ij}^+ = \beta_{ij}^c - \frac{\sigma_j^c}{y^T s}\sum_{k=1}^{n}\gamma^+\beta_{ik}^c - \frac{\sigma_i^c}{y^T s}\sum_{k=1}^{n}\gamma^+\beta_{jk}^c$$
$$-\sigma_i^c\sigma_j^c\frac{(s-H_c y)^T y}{(y^T s)^2}. \quad (10)$$

By combining a few terms and using (9), we get the simplification

$$\beta_{ij}^+ = \beta_{ij}^c - \sum_{k=1}^{n}\frac{\gamma_k^+}{y^T s}\left(\sigma_j^c\beta_{ik}^c + \sigma_i^c\beta_{jk}^c\right)$$
$$+\sigma_i^c\sigma_j^c\sum_{k=1}^{n}\sum_{p=1}^{n}\frac{((\gamma_k^+ - \gamma_k^c)\gamma_p^+}{(y^T s)^2}\beta_{kp}^c. \quad (11)$$

This can be implemented on a distributed memory machine, even with limited local memory (like the nCUBE/1). For example, since processor p only requires the n elements $\beta_{1p}^c, \beta_{2p}^c, \ldots, \beta_{np}^c$, we may avoid replicating the entire inverse Hessian approximation on each node by distributing the matrix by rows. Using the row distribution of the matrix, we develop Algorithm 1.1.

As a note, the *vm_concat* communication procedure allows data sharing over all of the nodes in the allocated subcube in logarithmic time. The standard implementation of the *vm_concat* operation is to combine using addition across the processors, with zeros in place of unknown quantities on each processor. (See [3] for more information on the *vm_concat* procedure.) The stopping criteria to be used can be studied as a subject area unto itself. As an example, $\|x_{k+1} - x_k\| \leq \epsilon$ should be included in the stopping criteria, where $\epsilon > 0$ is the allowable error in the solution.

2 Numerical Results

The initial implementation of Algorithm 1.1 was done in the C programming language on the 1024 node nCUBE/1 hypercube. In that implementation,

Algorithm 1.1: The Parallel Inverse BFGS Algorithm

Assume $f : \Omega \subset\subset \mathbf{R}^n \to \mathbf{R}$ is smooth ($f \in \mathcal{C}^\infty$). Let $g = \nabla f, x_0 \in \Omega, H^0$ be an approximation to $(\nabla^2 f(x_0))^{-1}$, and $\{b_j\}$ be an orthonormal basis for $\mathbf{R}^n$. We will refer to the coordinates as $0, 1, \ldots, n-1$ to conform to the array base in the C programming language.

(1) Set $\beta_{ij}^0 = b_j^T H^0 b_i$ for each $i, j = 0, \ldots, n-1$.
(2) Set $\xi_i^0 = b_i^T x_0$ for each $i = 0, \ldots, n-1$.
(3) Do in parallel $q = 0, \ldots, P-1$
  (A) Set $k = 0$ and define a task set $\Lambda = \Lambda(q)$.
  (B) For $p \in \Lambda$, evaluate $\gamma_p^k = [g(x_k)]_p$.
  (C) *vm_concat*$(\gamma_0^k, \ldots, \gamma_{n-1}^k)$.
  (D) Repeat
    (a) For $p \in \Lambda$, set $\xi_p^{k+1} = \xi_p^k - \sum_i \gamma_i^k \beta_{ip}^k$.
    (b) *vm_concat*$(\xi_0^{k+1}, \ldots, \xi_{n-1}^{k+1})$.
    (c) For $p \in \Lambda$, evaluate $\gamma_p^{k+1} = [g(x_{k+1})]_p$.
    (d) *vm_concat*$(\gamma_1^{k+1}, \ldots, \gamma_n^{k+1})$.
    (e) Set $\alpha = \sum_j (\gamma_j^{k+1} - \gamma_j^k)(\xi_j^{k+1} - \xi_j^k)$.
    (f) For $p \in \Lambda$, set $\mu_{2p} = \sum_j \gamma_j^k \beta_{jp}^k$.
    (g) For $p \in \Lambda$, set $\mu_{2p+1} = \sum_j \gamma_j^{k+1} \beta_{jp}^k$.
    (h) *vm_concat*$(\mu_0, \mu_1, \ldots, \mu_{2n-1})$.
    (i) Set $\tau = \sum_j \gamma_j^{k+1} (\mu_{2j+1} - \mu_{2j})$.
    (j) For $p \in \Lambda$, set $\sigma_p = \xi_p^{k+1} - \xi_p^k$.
    (k) For $i = 0, \ldots, n-1$
      (*i*) Set $\sigma_i = \xi_i^{k+1} - \xi_i^k$.
      (*ii*) For $p \in \Lambda$, set $\beta_{ip}^{k+1} = \beta_{ip}^k$
        $-\frac{1}{\alpha}(\sigma_p \mu_{2i+1} + \sigma_i \mu_{2p+1} - \sigma_i \sigma_p \tau)$.
    (l) Set $k = k + 1$.
  (E) Until stopping criteria satisfied.
  (F) Return x_{k+1} to host as x_{final}.
(4) Set $\hat{x}_* = \sum \xi_j^{final} b_j$.

the standard basis for $\mathbf{R}^n$, and the inverse of a finite difference approximation to the Hessian at x_0 were used, and the matrix was distributed to the processors by rows. The second implementation was done using the *Zipcode* message passing paradigm (see [5]) and the *Reactive Kernel* (see [4]) on the

192 node Symult S2010 mesh multicomputer.

The results from these initial implementation show that the algorithm is effective on convex functions. We present two simplified examples below of convex functions that arose as part of a maximum likelihood estimation.

2.1 Example 1: A Quadratic

First, we consider minimizing the following quadratic

$$f(x) = \frac{1.9}{n-1} \sum_{i=1}^{n} \sum_{j<i} (\xi_i - 1)(\xi_j - 1) + \sum_{i=1}^{n} (\xi_i - 1)^2.$$

As an initial guess, we chose the two's vector, $x_0 = (2, 2, \ldots, 2)$. The computation was stopped when $\|g(x_{k+1})\| < \sqrt{macheps}$, $\|x_{k+1} - x_k\| < \sqrt{macheps}$, or the number of iterations exceeded 500. The minimizer is the one's vector, $x_* = (1, 1, \ldots, 1)$. The tables below (Tables 2.1.1 and 2.1.2) present the numerical results obtained by solving the problem on the nCUBE/1 and the Symult S2010 using the inverse of a finite difference approximation to the initial Hessian. The timing data in the tables represent the the time taken on the slowest node averaged over several runs (average maximum time), and the standard deviation of the listed value is also given. In each case, the number of runs made was ten or greater.

By plotting the number of processors against the execution times, we observe that the performance curve becomes flat rather rapidly. This is not surprising as the evaluation of the quadratic is not very computationally intensive. It is interesting to note the small indentations in the Symult plot at the powers of two. These occur because the dimensional collapse algorithm used in the *vm_concat* incurs extra overhead when the number of processors is not a power of two.

2.2 Example 2: An Exponential Function

As another example, consider the exponential function

$$f(x) = \left(\sum_{i=1}^{n} \xi_i^2 \right) \left(\frac{1}{n^2} \sum_{i=1}^{n} e^{\xi_i^2} \right).$$

This time we chose the alternating vector $x_0 = (1, -1, \ldots, 1, -1)$ as the initial guess. The next tables present the results obtained solving the problem using the same stopping criteria as before ($\|g(x_{k+1})\| < \sqrt{macheps}$,

Table 2.1.1: Symult, 128 Quad., $B_0 \approx (\nabla^2 f(x_0))^{-1}$

P	Iter	Time(sec)	$f(\hat{x}_*)$	$\|\hat{x}_*\|_\infty$
1	2	4.276 ± 0.001	7.4e-22	1.0000
2	2	2.195 ± 0.000	7.4e-22	1.0000
4	2	1.149 ± 0.001	7.4e-22	1.0000
8	2	0.637 ± 0.001	7.4e-22	1.0000
10	2	0.606 ± 0.000	7.4e-22	1.0000
16	2	0.391 ± 0.000	7.4e-22	1.0000
20	2	0.455 ± 0.001	7.4e-22	1.0000
32	2	0.280 ± 0.000	7.4e-22	1.0000
50	2	0.359 ± 0.000	7.4e-22	1.0000
64	2	0.235 ± 0.001	7.4e-22	1.0000
80	2	0.368 ± 0.000	7.4e-22	1.0000
100	2	0.336 ± 0.000	7.4e-22	1.0000
128	2	0.224 ± 0.001	7.4e-22	1.0000

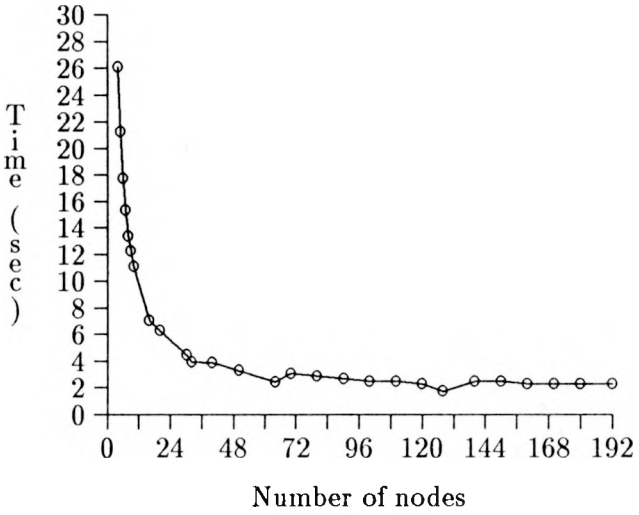
Table 2.1.2: nCUBE, 128 Quad., $B_0 \approx (\nabla^2 f(x_0))^{-1}$

P	Iter	Time(sec)	$f(\hat{x}_*)$	$\|\hat{x}_*\|_\infty$
1	2	9.430 ± 0.000	7.4e-22	1.0000
2	2	5.682 ± 0.001	7.4e-22	1.0000
4	2	3.857 ± 0.001	7.4e-22	1.0000
8	2	3.025 ± 0.003	7.4e-22	1.0000
16	2	2.682 ± 0.002	7.4e-22	1.0000
32	2	2.587 ± 0.010	7.4e-22	1.0000
64	2	2.593 ± 0.007	7.4e-22	1.0000
128	2	2.672 ± 0.001	7.4e-22	1.0000

$\|x_{k+1} - x_k\| < \sqrt{macheps}$, or the number of iterations exceeded 500), and selecting several different approximations for the initial inverse Hessian. The minimizer is the origin, $x_* = (0, 0, \ldots, 0)$.

The first four tables (Tables 2.2.1, 2.2.2, 2.2.3, 2.2.4) show the results for solving the $n = 128$ variable problem. Results obtained using the identity and scaled identity as the initial approximate inverse Hessian on the

Figure 2.1.1: Symult, 512 Quad., $B_0 \approx (\nabla^2 f(x_0))^{-1}$



nCUBE/1 are shown in Tables 2.2.1 and 2.2.2. Comparing these with 2.2.3 shows that there is a significant advantage in selecting a good approximation of the initial Hessian inverse. The fourth table (Table 2.2.4) shows the results obtained by using the Symult instead of the nCUBE/1.

Table 2.2.1: nCUBE, 128 Exp., $B_0 = I$

P	Iter	Time(sec)	$f(\hat{x}_*)$	$\|\hat{x}_*\|_\infty$
1	16	167.364 ± 0.000	1.1e-15	3.3e-08
2	16	86.199 ± 0.001	1.1e-15	3.3e-08
4	16	45.953 ± 0.001	1.1e-15	3.3e-08
8	16	26.383 ± 0.002	1.1e-15	3.3e-08
16	16	17.149 ± 0.001	1.1e-15	3.3e-08
32	16	13.078 ± 0.002	1.1e-15	3.3e-08
64	16	11.618 ± 0.088	1.1e-15	3.3e-08
128	16	11.419 ± 0.098	1.1e-15	3.3e-08

The next four tables are the analogs for the $n = 512$ variable problem. Again we see a similar pattern in Tables 2.2.5, 2.2.6, and 2.2.7. There is

Figure 2.1.2: nCUBE, 512 Quad., $B_0 \approx (\nabla^2 f(x_0))^{-1}$

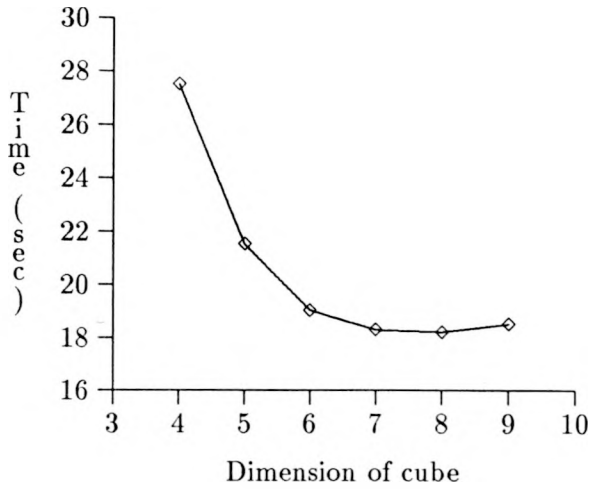


Table 2.2.2: nCUBE, 128 Exp., $B_0 = \frac{1}{|f(x_0)|} I$

P	Iter	Time(sec)	$f(\hat{x}_*)$	$\|\hat{x}_*\|_\infty$
1	17	178.015 ± 0.000	2.5e-15	5.0e-08
2	17	91.630 ± 0.001	2.5e-15	5.0e-08
4	17	48.798 ± 0.008	2.5e-15	5.0e-08
8	17	27.964 ± 0.003	2.5e-15	5.0e-08
16	17	18.137 ± 0.008	2.5e-15	5.0e-08
32	17	13.837 ± 0.086	2.5e-15	5.0e-08
64	17	12.193 ± 0.002	2.5e-15	5.0e-08
128	17	11.977 ± 0.002	2.5e-15	5.0e-08

a significant savings associated with using the better approximation of the initial Hessian inverse.

3 Performance Analysis

For those who solve scientific applications, the most fundamental performance measure is solution time. Thus, we seek to study indicators that

Table 2.2.3: nCUBE, 128 Exp., $B_0 \approx (\nabla^2 f(x_0))^{-1}$

P	Iter	Time(sec)	$f(\hat{x}_*)$	$\|\hat{x}_*\|_\infty$
1	12	127.125 ± 0.000	1.2e-15	3.7e-08
2	12	65.633 ± 0.002	1.2e-15	3.7e-08
4	12	35.145 ± 0.002	1.2e-15	3.7e-08
8	12	20.322 ± 0.002	1.2e-15	3.7e-08
16	12	13.358 ± 0.081	1.2e-15	3.7e-08
32	12	10.233 ± 0.002	1.2e-15	3.7e-08
64	12	9.086 ± 0.001	1.2e-15	3.7e-08
128	12	8.927 ± 0.002	1.2e-15	3.7e-08

Table 2.2.4: Symult, 128 Exp., $B_0 \approx (\nabla^2 f(x_0))^{-1}$

P	Iter	Time(sec)	$f(\hat{x}_*)$	$\|\hat{x}_*\|_\infty$
1	17	51.301 ± 0.000	1.8e-18	1.3e-09
2	17	26.153 ± 0.001	1.8e-18	1.3e-09
4	17	13.474 ± 0.001	1.8e-18	1.3e-09
8	17	7.211 ± 0.000	1.8e-18	1.3e-09
10	17	6.536 ± 0.001	1.8e-18	1.3e-09
16	17	4.163 ± 0.000	1.8e-18	1.3e-09
20	17	4.499 ± 0.001	1.8e-18	1.3e-09
32	17	2.726 ± 0.000	1.8e-18	1.3e-09
50	17	3.180 ± 0.001	1.8e-18	1.3e-09
64	17	2.089 ± 0.000	1.8e-18	1.3e-09
80	17	3.099 ± 0.001	1.8e-18	1.3e-09
100	17	2.706 ± 0.001	1.8e-18	1.3e-09
128	17	1.852 ± 0.001	1.8e-18	1.3e-09

predict lower solution times. Clearly, efficiency and speedup are not such indicators. Although high efficiency and near-linear speedup are usually assumed to accompany minimal execution times, by including additional, but unnecessary, work we increase the speedup and the efficiency while increasing the solution time. Best sequential time based measures do not account for machine effects. For example, the use of many processors might enable

Table 2.2.5: nCUBE, 512 Exp., $B_0 = I$

P	Iter	Time(sec)	$f(\hat{x}_*)$	$\|\hat{x}_*\|_\infty$
16	16	196.573 ± 0.084	1.0e-14	1.0e-07
32	16	117.871 ± 0.008	1.0e-14	1.0e-07
64	16	80.723 ± 0.006	1.0e-14	1.0e-07
128	16	64.766 ± 0.278	1.0e-14	1.0e-07
256	16	58.545 ± 0.312	1.0e-14	1.0e-07
512	16	57.386 ± 0.103	1.0e-14	1.0e-07

Table 2.2.6: nCUBE, 512 Exp., $B_0 = \frac{1}{|f(x_0)|} I$

P	Iter	Time(sec)	$f(\hat{x}_*)$	$\|\hat{x}_*\|_\infty$
16	15	185.601 ± 0.014	1.9e-14	1.4e-07
32	15	111.525 ± 0.004	1.9e-14	1.4e-07
64	15	76.481 ± 0.007	1.9e-14	1.4e-07
128	15	61.506 ± 0.261	1.9e-14	1.4e-07
256	15	55.252 ± 0.094	1.9e-14	1.4e-07
512	15	54.348 ± 0.010	1.9e-14	1.4e-07

the working data to be located wholly in cache thereby providing higher performance than predicted.

Instead of these measures, we choose to use granularity, which is the ratio of time spent in necessary communication and time necessary to create those messages. In layman's terms, we would define granularity as the time spent in useful communication divided by the time spent in useful computation. Since this measure is heavily dependent upon the hardware and software alike, we refer to the granularity of an algorithm with respect to a machine.

To compute the granularity, we use the formula

$$\Gamma = \frac{T_{comm}}{T_{calc}}.$$

By inspecting the algorithm presented in the previous section, we observe that all of the communication time is spent in the *vm_concat* operation.

Table 2.2.7: nCUBE, 512 Exp., $B_0 \approx (\nabla^2 f(x_0))^{-1}$

P	Iter	Time(sec)	$f(\hat{x}_*)$	$\|\hat{x}_*\|_\infty$
16	12	151.432 ± 0.006	1.2e-15	3.4e-08
32	12	91.722 ± 0.100	1.2e-15	3.4e-08
64	12	63.436 ± 0.004	1.2e-15	3.4e-08
128	12	51.604 ± 0.417	1.2e-15	3.4e-08
256	12	46.610 ± 0.243	1.2e-15	3.4e-08
512	12	45.694 ± 0.093	1.2e-15	3.4e-08

Table 2.2.8: Symult, 512 Exp., $B_0 \approx (\nabla^2 f(x_0))^{-1}$

P	Iter	Time(sec)	$f(\hat{x}_*)$	$\|\hat{x}_*\|_\infty$
4	14	171.759 ± 0.003	5.8e-19	7.6e-10
8	14	87.246 ± 0.002	5.8e-19	7.6e-10
10	14	71.846 ± 0.005	5.8e-19	7.6e-10
16	14	45.204 ± 0.002	5.8e-19	7.6e-10
20	14	39.113 ± 0.002	5.8e-19	7.6e-10
32	14	24.408 ± 0.001	5.8e-19	7.6e-10
50	14	18.855 ± 0.002	5.8e-19	7.6e-10
64	14	14.244 ± 0.002	5.8e-19	7.6e-10
80	14	15.770 ± 0.002	5.8e-19	7.6e-10
100	14	13.152 ± 0.003	5.8e-19	7.6e-10
128	14	9.394 ± 0.001	5.8e-19	7.6e-10
150	14	12.678 ± 0.002	5.8e-19	7.6e-10
170	14	11.425 ± 0.002	5.8e-19	7.6e-10
192	14	11.383 ± 0.011	5.8e-19	7.6e-10

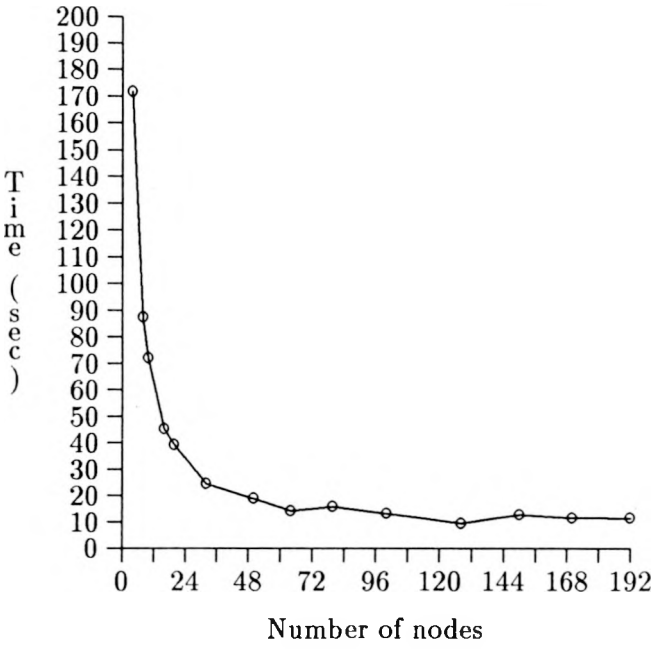
Thus the communication time is

$$T_{comm} = 2T_{combine}(8n) + T_{combine}(16n)$$

But, by experimental testing and data-fitting, we can write

$$T_{combine}(L) = (\gamma_1 + \delta_1 L)\log_2 p + (\gamma_2 + \delta_2 L)$$

Figure 2.2.3: Symult, 512 Exp., $B_0 \approx (\nabla^2 f(x_0))^{-1}$



where $\gamma_1, \delta_1, \gamma_2, \delta_2$ are fit parameters representing the latency and per-byte costs of the communication ([6]). Note that these fit parameters are machine dependent.

Again examining the algorithm, we can write the calculation time as

$$T_{calc} = \tau \Omega_{math} + T_{fcn}$$

where τ is the machine dependent parameter representing the time to do a double precision math operation, Ω_{math} is the math operation complexity of the algorithm, and T_{fcn} is the time to perform a function evaluation (or the time required to evaluate one component of the gradient). Tables 3.1 and 3.2 show the physical constants for the nCUBE/1 and the Symult S2010, respectively. Table 3.3 gives the operation counts for the algorithm.

Using the data from these tables, we determine

$$T_{combine} = (3\gamma_1 + 32n\delta_1)\log_2 p + 3\gamma_2 + 32n\delta_2$$

$$T_{calc} = (14nm - m + 13n - 3 + 4n\log_2 p)\tau + T_{fcn}$$

Figure 2.2.4: nCUBE, 512 Exp., $B_0 \approx (\nabla^2 f(x_0))^{-1}$

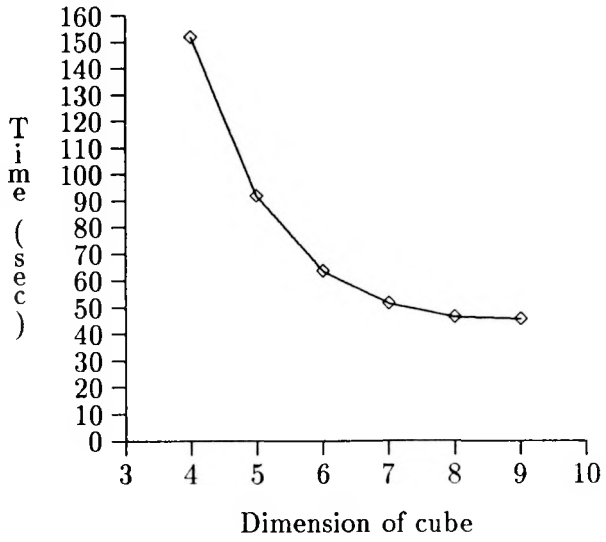


Table 3.1: Physical Constants: nCUBE/1

message latency ($\gamma_1 = \gamma_2$)	390.00 (μs)
transfer time ($\delta_1 = \delta_2$)	5.00 (μs)
arithmetic time	$\tau = 6.70$ (μs)

If we use analytic derivatives for the gradients, we can rewrite the quadratic from example 1 above as

$$g_i(x) = \frac{1.9}{n-1} \sum_j \xi_j - (\frac{1.9}{n-1} - 2)\xi_i$$

Table 3.2: Physical Constants: Symult S2010

message latency	$\gamma_1 = 607.66, \gamma_2 = 299.94$ (μs)
transfer time	$\delta_1 = 0.4398, \delta_2 = 0.2756$ (μs)
arithmetic time	$\tau = 1.25$ (μs)

Table 3.3: Complexity counts: PiBFGS Algorithm

fcn evals	m gradient
arithmetic ops	$14nm - m + 13n - 3 + 4n\log_2 p$
communication	$2\log_2 p$ @ $8n$ bytes
	$\log_2 p$ @ $16n$ bytes

which gives an evaluation time of

$$T_{quad} = (n+5)m\tau.$$

Similarly, the derivative of the exponential in example 2 can be computed by

$$g_i(x) = 2\xi_i \sum_j \left(\frac{e^{\xi_j^2}}{n^2} + \frac{\xi_j^2}{n^2} \right)$$

which gives an evaluation time of

$$T_{exp} = (9n+3)m\tau.$$

Combining the above formulas with the function evaluation times we can produce the following two tables. The first table (Table 3.4) gives the granularity of the PiBFGS Algorithm on the nCUBE/1 solving the $n = 128$ variable problems. The second table (Table 3.5) gives the granularity of the same algorithm on the Symult S2010. Tables 3.6 and 3.7 give the granularity of the PiBFGS algorithm on the nCUBE/1 and Symult, respectively, solving the $n = 512$ variable problems. By default the granularity of the algorithm on 1 processor of any machine is 0.

Note that the PiBFGS algorithm on the nCUBE/1 is much coarser grain than the PiBFGS algorithm on the Symult. (Actually, we should expect this because the nCUBE/1 has much slower communication than the Symult.) By inspecting the granularity of the algorithm on two differing machines, we would determine that the PiBFGS algorithm is better suited to the Symult As a natural extension, fixing the machine to be used we can compare two possible algorithms using granularities; the algorithm with the lower granularity on the given machine is the better of the two since it spends more time in computation than the other.

Another useful piece of information that we can learn from the granularity of an algorithm on a machine involves the graph of the granularity curve.

Table 3.4: Granularity of PIBFGS on NCUBE/1, $n = 128$

P	Granularity (Quad)	Granularity (Exp)
1	0.0000	0.0000
2	8.9184	10.9060
4	12.1015	14.4754
8	14.7301	17.3114
16	16.9376	19.6200
32	18.8176	21.5365
64	20.4380	23.1531
128	21.8491	24.5355

Table 3.5: Granularity of PIBFGS on Symult S2010, $n = 128$

P	Granularity (Quad)	Granularity (Exp)
1	0.0000	0.0000
2	1.3687	1.4258
4	1.9984	2.0720
8	2.5031	2.5855
16	2.9166	3.0033
32	3.2615	3.3500
64	3.5536	3.6423
128	3.8042	3.8921

If we plot the data, we observe that the granularity curve is flat where the execution time curve is flat (in other words, the slopes of the two graphs are directly related). Even by looking at the tables instead of the graphs, we can observe this relationship: where the granularity is increasing slowly as more processors are added, the execution time is decreasing slowly.

Looking at Tables 3.6 and 3.7, we again observe that the PiBFGS algorithm is better run on the Symult than the nCUBE/1. Note that the granularity of the PiBFGS on the Symult for the $n = 128$ variable problems is higher than that for the $n = 512$ indicating that more execution time is spent in computation for the larger problems. However, because of the slow communication on the nCUBE/1, the situation there is reversed.

Table 3.6: Granularity of PIBFGS on NCUBE/1, $n = 512$

P	Granularity (Quad)	Granularity (Exp)
1	0.0000	0.0000
2	8.5961	10.4686
4	11.6590	13.9051
8	14.1865	16.6363
16	16.3075	18.8595
32	18.1129	20.7044
64	19.6683	22.2601
128	21.0222	23.5898
256	22.2113	24.7393
512	23.2641	25.7429

Table 3.7: Granularity of PIBFGS on Symult S2010, $n = 512$

P	Granularity (Quad)	Granularity (Exp)
1	0.0000	0.0000
2	0.8746	0.9102
4	1.2645	1.3100
8	1.5769	1.6278
16	1.8328	1.8865
32	2.0463	2.1011
64	2.2272	2.2821
128	2.3823	2.4367
256	2.5168	2.5704
512	2.6345	2.6871

4 Conclusions and Future Research

The above numerical results indicate that the PiBFGS Algorithm is effective in solving convex problems. This is especially true when function evaluations are expensive, or the problem sizes are extremely large. However, there are several issues which must be addressed if this method is to be used in "real world" applications.

For very large problems the question of storing the matrix B becomes the dominant concern. Thus we need to develop a reduced storage version of the above algorithm which does not require the storage of the matrix.

Another issue relates to the sensitivity of the algorithm to its initial data. To improve robustness, we need to add a line search strategy which admits to parallel use and does not incur unnecessary function evaluations.

We know that the choice of basis used in the PiBFGS algorithm does not affect the rate of convergence (see [9] and [8]). Therefore, the basis to be used can be chosen to satisfy other criteria: for example, to enhance robustness or to reduce the amount of linear algebra done.

Finally, one of the most important issues to be addressed is data distribution independence. Since optimization problems usually arise as part of a larger scientific application (for example, optimization of a chemical plant), the optimization routine should be able to utilize many data distributions. Particularly in larger problems, data redistribution is prohibitively costly, so compromises must be made in order to achieve high performance for the application as a whole and not just the optimization step. The PiBFGS algorithm can be easily modified to meet this objective.

Acknowledgements

I would like to thank the University of South Carolina and the California Institute of Technology for access to their machines, Chuck Baldwin for the communication subroutines on the nCUBE/1, and especially Tony Skjellum for *Zipcode* and his invaluable assistance with performance measures.

References

[1] R. Byrd, R. Schnabel, and G. Shultz, "Parallel Quasi–Newton Methods for Unconstrained Optimization," *Mathematical Programming* 42, pp. 273–306, 1988.

[2] J. Dennis, and R. Schnabel, *Numerical Methods for Unconstrained Optimization and Nonlinear Equations,* Prentice-Hall, 1983.

[3] G. Fox, M. Johnson, G. Lyzenga, S. Otto, J. Salmon, and D. Walker, *Solving Problems on Concurrent Computers, Vol I,* Prentice-Hall, 1988.

[4] C. L. Seitz, J. Seizovic, W-K. Su, *The C Programmer's Abbreviated Guide to Multicomputer Programming*, Caltech Computer Science Technical Report Caltech-CS-TR-88-1, 1988.

[5] A. Skjellum, and M. Morari, *Zipcode: A Portable Communication Layer for High Performance Multicomputing - Practice and Experience*, Technical Report UCRL-JC-106725, March 1991. Submitted to *Concurrency: Practice & Experience*.

[6] A. Skjellum, *Concurrent Dynamic Simulation: Multicomputer Algorithms Research Applied to Ordinary Differential-Algebraic Process Systems in Chemical Engineering*, Ph.D. Thesis, California Institute of Technology, 1990.

[7] C. H. Still, "Parallel Quasi-Newton Methods for Unconstrained Optimization," *Proceedings of the Fifth Distributed Memory Computing Conference*, pp 263–271, 1990.

[8] C. H. Still, *Parallel Methods for Unconstrained Optimization*, Ph.D. Thesis, University of South Carolina, 1990.

[9] C. H. Still, and G. W. Johnson, "Parallel Quasi-Newton Methods for Unconstrained Minimization", (in prep.), 1991.

Technical Information Department · Lawrence Livermore National Laboratory
University of California · Livermore, California 94551